BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA TOÁN - ỨNG DỤNG

BÁO CÁO ĐỒ ÁN CUỐI KỲ

Học phần: Xử lý dữ liệu lớn (Big Data)

CHỦ ĐỀ: XÂY DỰNG HỆ THỐNG SOC PHÂN TÍCH DỮ LIỆU LỚN VỚI KAFKA VÀ SPARK

Giảng viên hướng dẫn: Vũ Ngọc Thanh Sang

Sinh viên thực hiện : Nguyễn Trương Cao Sơn

Lớp : DDU1231

TP Hồ Chí Minh, năm 2024

Mục lục

LỜI MỞ ĐẦU **1**

1 TỔNG QUAN VỀ ĐỀ TÀI **2**
1.1 Giới thiệu về đề tài 2
1.1.1 Bối cảnh nghiên cứu 2
1.1.2 Các khái niệm và công nghệ nền tảng 2
1.1.3 Lý do chọn đề tài 3
1.2 Mục tiêu và phạm vi nghiên cứu 3
1.2.1 Mục tiêu nghiên cứu 3
1.2.2 Phạm vi giới hạn của đồ án 3
1.3 Câu hỏi nghiên cứu 4
1.4 Các hệ thống và giải pháp tương đương (Related Works) 4
1.5 Ý nghĩa thực tiễn 4

2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ **6**
2.1 Khái quát về Xử lý dữ liệu lớn 6
2.1.1 Đặc trưng của dữ liệu lớn 6
2.1.2 Xử lý theo lô và Xử lý theo luồng 6
2.2 Nền tảng phân phối sự kiện Apache Kafka 7
2.2.1 Kiến trúc kỹ thuật của Kafka 7
2.2.2 Cơ chế phân vùng và nhóm tiêu thụ 7
2.3 Khung tính toán phân tán Apache Spark 7
2.3.1 Cơ chế tính toán trực tiếp trên bộ nhớ 7
2.3.2 Xử lý luồng có cấu trúc và Khung thời gian 7
2.4 Ứng dụng trong An toàn thông tin 8
2.4.1 Khái niệm về hệ thống quản lý sự kiện bảo mật 8
2.4.2 Cơ chế phát hiện tấn công dò đoán mật khẩu 8

3 ỨNG DỤNG VÀ TRIỂN KHAI BÀI TOÁN THỰC TẾ **9**
3.1 Phương pháp nghiên cứu và kiến trúc hệ thống 9
3.1.1 Sơ đồ khối kiến trúc tổng thể 9
3.1.2 Lựa chọn công nghệ triển khai ảo hóa 10
3.2 Thu thập và giả lập dữ liệu 10
3.2.1 Xây dựng kịch bản mạng máy tính ma tấn công 10
3.2.2 Cấu hình nhà sản xuất dữ liệu Kafka 11
3.3 Xử lý và phân tích dữ liệu 11
3.3.1 Xử lý luồng sự kiện với khung tính toán Spark 11
3.3.2 Thuật toán phát hiện bất thường và trích xuất danh sách chặn . . . 12

4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ **14**

4.1 Kết quả chạy thực nghiệm 14
4.1.1 Giao diện sinh dữ liệu log 14
4.1.2 Bảng cảnh báo thời gian thực và File kết quả 14
4.2 Đưa ra nhận xét và đánh giá 14
4.2.1 Ưu điểm của hệ thống 14
4.2.2 Hạn chế và hướng phát triển 14

KẾT LUẬN **15**

LỜI CẢM ƠN **16**

TÀI LIỆU THAM KHẢO **17**

Danh sách hình vẽ

3.1 Sơ đồ khối kiến trúc tổng thể của hệ thống 9

LỜI MỞ ĐẦU

(Viết lời mở đầu của bạn vào đây...)

Chương 1

TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu về đề tài

1.1.1 Bối cảnh nghiên cứu

Trong kỷ nguyên số hóa hiện nay, các ứng dụng web và nền tảng thương mại điện tử đang phát triển bùng nổ, kéo theo lượng dữ liệu giao dịch và lưu lượng truy cập khổng lồ được sinh ra mỗi ngày. Sự phát triển này cũng biến các hệ thống trực tuyến trở thành mục tiêu béo bở cho tội phạm mạng. Các hình thức tấn công diễn ra ngày càng tinh vi, điển hình như dò quét lỗ hổng, thu thập dữ liệu trái phép, hay phổ biến nhất là tấn công dò đoán mật khẩu thông qua các mạng lưới máy tính ma hay còn gọi là Botnet.

Trước đây, quản trị viên an toàn thông tin thường sử dụng các công cụ phân tích nhật ký hệ thống truyền thống hoặc xử lý dữ liệu theo từng đợt. Tuy nhiên, khi đối mặt với lượng dữ liệu lớn lên tới hàng chục nghìn yêu cầu mỗi giây, các hệ thống cũ nhanh chóng bộc lộ điểm yếu: quá tải, nghẽn cổ chai và độ trễ cao. Việc phát hiện một cuộc tấn công sau vài giờ, hay thậm chí vài phút, là quá muộn để ngăn chặn rủi ro rò rỉ dữ liệu hoặc sập hệ thống.

Thực tiễn đó đặt ra yêu cầu cấp thiết phải xây dựng một Trung tâm Điều hành An ninh thế hệ mới, gọi tắt là hệ thống SOC. Hệ thống này phải có khả năng tiếp nhận và phân tích dữ liệu nhật ký hệ thống theo thời gian thực để chặn đứng các mối đe dọa ngay khi chúng vừa khởi phát.

1.1.2 Các khái niệm và công nghệ nền tảng

Để làm rõ hướng giải quyết cho bài toán nêu trên, phần này sẽ giải thích bản chất của các khái niệm và công nghệ cốt lõi được sử dụng xuyên suốt đồ án:

- **Nhật ký hệ thống (Log):** Là các tệp dữ liệu tự động ghi lại mọi hoạt động xảy ra trên máy chủ (ai vừa truy cập, địa chỉ IP là gì, hành động thành công hay thất bại). Đây chính là nguồn "nguyên liệu"đầu vào để hệ thống tiến hành phân tích.

- **Mạng máy tính ma (Botnet) và Tấn công dò đoán mật khẩu (Brute Force):** Kẻ tấn công thường khống chế hàng ngàn máy tính của người dùng bình thường tạo thành một mạng lưới (Botnet). Chúng điều khiển mạng lưới này cùng lúc gửi hàng triệu lượt thử mật khẩu sai (Brute Force) nhằm làm sập hệ thống hoặc đánh cắp tài khoản.

- **Apache Kafka (Trạm trung chuyển dữ liệu):** Về bản chất, Kafka là một hệ thống điều phối và truyền tải dữ liệu tốc độ cao. Khi website bị tấn công, dữ liệu nhật ký sinh ra ồ ạt như một cơn lũ. Nếu đổ trực tiếp lượng dữ liệu này vào hệ thống phân tích, máy chủ chắc chắn sẽ quá tải. Kafka ra đời để đóng vai trò như một "hồ chứa nước đệm". Nó hứng chịu toàn bộ lưu lượng khổng lồ từ máy chủ web, lưu trữ tạm thời một cách an toàn, rồi điều tiết dòng chảy đó sang cho các bộ phận xử lý phía sau với tốc độ phù hợp, giúp toàn bộ kiến trúc không bao giờ bị nghẽn.

- **Apache Spark (Khung tính toán trực tiếp trên bộ nhớ):** Nếu Kafka là người vận chuyển, thì Spark chính là "bộ não"xử lý. Đặc điểm làm nên sức mạnh của Spark là khả năng tính toán trực tiếp trên bộ nhớ RAM thay vì phải ghi và đọc dữ liệu liên tục từ ổ cứng như các công nghệ cũ. Nhờ vậy, Spark có thể hút dữ liệu liên tục từ Kafka, thực hiện đếm số lượt truy cập, gom nhóm các địa chỉ IP và phát hiện ra các điểm bất thường gần như ngay lập tức.

1.1.3 Lý do chọn đề tài

Từ những khái niệm trên, có thể thấy việc ứng dụng các công nghệ xử lý dữ liệu lớn vào bài toán an toàn thông tin là một hướng đi thiết thực và đầy tiềm năng. Sự kết hợp hoàn hảo giữa khả năng chịu tải tuyệt vời của Kafka và tốc độ xử lý tức thời của Spark chính là lời giải trọn vẹn cho bài toán độ trễ trong giám sát hệ thống.

Hơn nữa, một luồng dữ liệu nhật ký không chỉ có giá trị về mặt phát hiện mã độc mà còn chứa đựng vô vàn thông tin hữu ích về hành vi người dùng, phục vụ cho công tác phân tích và thống kê của doanh nghiệp. Do đó, đề tài **"Xây dựng hệ thống SOC phân tích dữ liệu lớn với Kafka và Spark"** được lựa chọn nhằm kết hợp kiến thức chuyên ngành vào một bài toán thực tế, mang lại góc nhìn đa chiều: vừa bảo vệ hệ thống nền tảng, vừa tận dụng triệt để nguồn dữ liệu sẵn có.

1.2 Mục tiêu và phạm vi nghiên cứu

1.2.1 Mục tiêu nghiên cứu

Đồ án được thực hiện với các mục tiêu cụ thể sau:

- **Về mặt lý thuyết:** Nghiên cứu và hiểu rõ kiến trúc, nguyên lý hoạt động của hệ sinh thái dữ liệu lớn hiện đại, đặc biệt là luồng xử lý thời gian thực thông qua Kafka và Spark.
- **Về mặt thực hành:** Tự xây dựng và triển khai thành công một luồng dữ liệu thông suốt từ khâu thu thập, xử lý trực tiếp đến khâu trực quan hóa lên bảng điều khiển.
- **Về mặt ứng dụng:** Xây dựng thuật toán phát hiện bất thường dựa trên ngưỡng thống kê để tự động nhận diện các địa chỉ IP có hành vi tấn công và đưa vào danh sách chặn.

1.2.2 Phạm vi giới hạn của đồ án

Do giới hạn về thời gian và tài nguyên phần cứng, đồ án tập trung vào các phạm vi sau:

- **Dữ liệu đầu vào:** Sử dụng dữ liệu nhật ký máy chủ web được giả lập liên tục theo thời gian thực, bao gồm cả lưu lượng người dùng bình thường và các kịch bản tấn công.
- **Mô hình triển khai:** Toàn bộ hệ thống Kafka và Spark được triển khai cục bộ thông qua nền tảng ảo hóa Docker để đảm bảo tính đồng nhất và dễ dàng đóng gói.
- **Phương pháp phân tích:** Đồ án sử dụng phương pháp thống kê và phân tích tập luật cơ bản để phát hiện tấn công thay vì áp dụng các mô hình học máy phức tạp.

- **Trực quan hóa:** Giao diện giám sát được phát triển bằng ngôn ngữ Python kết hợp với các thư viện hỗ trợ vẽ biểu đồ động.

1.3 Câu hỏi nghiên cứu

Để bám sát các mục tiêu đã đề ra và làm cơ sở đánh giá tính hiệu quả của hệ thống, đồ án tập trung giải quyết các câu hỏi nghiên cứu trọng tâm sau:

- **Khả năng chịu tải (Throughput):** Nền tảng Apache Kafka có khả năng tiếp nhận và điều phối ổn định lưu lượng nhật ký truy cập (log traffic) lên tới bao nhiêu yêu cầu mỗi giây (requests/second) trước khi xảy ra hiện tượng mất mát dữ liệu?
- **Độ trễ xử lý (Latency):** Khung tính toán Apache Spark Structured Streaming có thể rút ngắn thời gian phát hiện dấu hiệu tấn công từ lúc log được sinh ra cho đến khi đưa IP vào danh sách chặn xuống mức bao nhiêu giây?
- **Ngưỡng phát hiện (Thresholding):** Làm thế nào để xác định được cửa sổ thời gian (Time Window) và ngưỡng giới hạn số lỗi truy cập (Error Rate Threshold) phù hợp để phân biệt chính xác giữa một cuộc tấn công Brute Force/Botnet và các truy cập lỗi thông thường của người dùng thực?

1.4 Các hệ thống và giải pháp tương đương (Related Works)

Trên thực tế, bài toán quản lý và phân tích log đã được nhiều hệ thống SIEM truyền thống giải quyết, điển hình như ELK Stack (Elasticsearch, Logstash, Kibana) hay Splunk. Các hệ thống này cung cấp khả năng lưu trữ và tìm kiếm dữ liệu toàn văn bản cực kỳ mạnh mẽ.

Tuy nhiên, nhược điểm cốt lõi của các giải pháp này là chủ yếu dựa trên cơ chế đánh chỉ mục (Indexing-based) dữ liệu tĩnh. Nghĩa là dữ liệu phải được ghi xuống ổ cứng và lập chỉ mục xong thì mới có thể chạy truy vấn cảnh báo. Khi đối mặt với kịch bản bị tấn công từ chối dịch vụ (DDoS) hoặc Botnet cường độ cao, cơ chế ghi ổ cứng liên tục này sẽ tạo ra nút thắt cổ chai (Bottleneck), khiến hệ thống có độ trễ lớn và thậm chí tự sập trước khi kịp phát ra cảnh báo.

Việc ứng dụng kiến trúc Kafka kết hợp Spark Streaming trong đồ án này mang lại một hướng tiếp cận khác biệt: **Xử lý dữ liệu đang bay (Data in-motion)**. Hệ thống không cần ghi dữ liệu thô xuống đĩa mà thực hiện gom nhóm và đếm trực tiếp trên bộ nhớ RAM (In-Memory Micro-batching), từ đó khắc phục triệt để nhược điểm độ trễ của các công cụ truyền thống trong các kịch bản tấn công khẩn cấp.

1.5 Ý nghĩa thực tiễn

Đồ án cung cấp một mô hình thu nhỏ nhưng bao hàm đầy đủ các thành phần cốt lõi của một nền tảng dữ liệu lớn thực tế. Kết quả của đồ án mang lại những giá trị rõ nét trên ba phương diện:

- **Về mặt vận hành an ninh:** Hệ thống chứng minh khả năng thay thế các công cụ rà soát lỗi thủ công vốn có độ trễ cao. Nhờ phân tích dữ liệu trực tiếp trên bộ nhớ,

hệ thống giúp đội ngũ bảo mật tự động nhận diện và trích xuất danh sách các địa chỉ IP độc hại gần như tức thời. Đây là tiền đề quan trọng để tích hợp với hệ thống tường lửa, giúp tự động chặn đứng tin tặc từ sớm.

- **Về mặt tận dụng dữ liệu thống kê:** Đồ án khẳng định sự đa dụng của nền tảng phân tích luồng. Cùng một nguồn dữ liệu đầu vào, ngoài việc phát hiện mã độc, hệ thống có thể đồng thời trích xuất các thông tin về lưu lượng truy cập hay tỷ lệ lỗi máy chủ. Quản trị viên có thể theo dõi sức khỏe website liên tục mà không gây thêm áp lực xử lý cho cơ sở dữ liệu chính yếu của doanh nghiệp.

- **Về mặt kiến trúc công nghệ:** Việc sử dụng trạm trung chuyển Kafka giúp tách biệt hoàn toàn cụm máy chủ web (nơi sinh dữ liệu) và cụm phân tích phía sau. Nhờ thiết kế phân tán này, hệ thống có khả năng mở rộng linh hoạt và chịu lỗi cực tốt, duy trì sự hoạt động nhịp nhàng ngay cả khi đối mặt với lượng truy cập tăng đột biến.

Chương 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Khái quát về Xử lý dữ liệu lớn

2.1.1 Đặc trưng của dữ liệu lớn

Dữ liệu lớn không chỉ đơn thuần là sự gia tăng về mặt dung lượng lưu trữ, mà còn bao hàm sự phức tạp trong khâu thu thập và phân tích. Theo mô hình tiêu chuẩn, một hệ thống dữ liệu lớn được định hình qua năm đặc trưng cốt lõi:

- **Dung lượng:** Khối lượng dữ liệu khổng lồ sinh ra liên tục. Đối với hệ thống giám sát an ninh, đây là hàng triệu dòng nhật ký truy cập mạng sinh ra mỗi ngày.
- **Tốc độ:** Yêu cầu luồng dữ liệu phải được tiếp nhận và xử lý tức thời để đáp ứng các bài toán cần phản ứng nhanh như phát hiện tấn công.
- **Đa dạng:** Dữ liệu đến từ nhiều nguồn máy chủ, thiết bị mạng khác nhau với các định dạng không đồng nhất như văn bản thuần, JSON hay định dạng mở rộng XML.
- **Tính xác thực:** Đòi hỏi độ tin cậy và tính toàn vẹn của dữ liệu đầu vào. Việc đảm bảo tính xác thực giúp hệ thống phân tích tránh được các cảnh báo sai hoặc nhận diện nhầm người dùng hợp lệ thành tin tặc.
- **Giá trị:** Là quá trình chắt lọc, biến đổi một luồng nhật ký hệ thống khô khan thành các cảnh báo bảo mật có giá trị thực tiễn cao.

2.1.2 Xử lý theo lô và Xử lý theo luồng

Trước khi đi sâu vào các kiến trúc dữ liệu, cần phân biệt rõ hai mô hình tính toán nền tảng hiện nay:

- **Xử lý theo lô (Batch Processing):** Là phương pháp tổng hợp và tích lũy dữ liệu thành các tập hợp lớn trong một chu kỳ thời gian nhất định (ví dụ: một giờ, một ngày). Sau khi kết thúc chu kỳ, hệ thống mới tiến hành xử lý toàn bộ khối lượng dữ liệu đó cùng một lúc.
- **Xử lý theo luồng (Stream Processing):** Là phương pháp xử lý dữ liệu liên tục. Hệ thống sẽ tiếp nhận và thực hiện tính toán ngay lập tức trên từng bản ghi (hoặc các lô dữ liệu cực nhỏ) ngay tại thời điểm chúng vừa được sinh ra.

Trong kiến trúc dữ liệu truyền thống, phương pháp xử lý theo lô thường được ưu tiên áp dụng. Dữ liệu được lưu trữ tĩnh trong các cơ sở dữ liệu và truy vấn định kỳ. Phương pháp này rất hiệu quả trong việc kết xuất báo cáo tổng kết, nhưng lại thiếu tính đáp ứng thời gian thực khi đối mặt với các cuộc tấn công mạng đang diễn ra.

Để khắc phục độ trễ hệ thống, các nền tảng giám sát hiện đại bắt buộc phải chuyển dịch sang phương pháp xử lý theo luồng. Đối với bài toán an ninh mạng, xử lý luồng giúp giảm thiểu độ trễ từ mức hàng giờ xuống chỉ còn vài phần nghìn giây. Điều này cho phép hệ thống nhận diện và cảnh báo tức thời các dấu hiệu bất thường, mang ý nghĩa quyết định trong việc ngăn chặn kịp thời các sự cố rò rỉ hoặc ngưng trệ dịch vụ.

2.2 Nền tảng phân phối sự kiện Apache Kafka

2.2.1 Kiến trúc kỹ thuật của Kafka

Apache Kafka được xây dựng dựa trên kiến trúc phân tán với khả năng mở rộng hệ thống theo chiều ngang. Cốt lõi của Kafka là một cụm máy chủ bao gồm nhiều trạm trung gian làm nhiệm vụ điều phối luồng thông điệp.

Mô hình giao tiếp của Kafka tuân theo cơ chế Xuất bản và Đăng ký. Điểm tạo nên tốc độ cực cao của Kafka so với các hệ thống hàng đợi truyền thống là khả năng ghi dữ liệu trực tiếp xuống ổ cứng một cách tuần tự, kết hợp với việc tận dụng tối đa bộ nhớ đệm của hệ điều hành. Cơ chế này cho phép Kafka tiếp nhận hàng triệu tin nhắn mỗi giây mà vẫn đảm bảo luồng dữ liệu không bị nghẽn và không bị mất mát khi xảy ra sự cố.

2.2.2 Cơ chế phân vùng và nhóm tiêu thụ

Để đảm bảo tốc độ đọc ghi song song, luồng thông điệp trong Kafka được tổ chức chặt chẽ theo các chủ đề và chia nhỏ thành nhiều phân vùng. Dữ liệu khi ghi vào phân vùng mang tính bất biến và được đánh số thứ tự để hệ thống dễ dàng theo dõi.

Đặc biệt, Kafka cung cấp cơ chế Nhóm tiêu thụ để phân tán khối lượng công việc. Nhiều máy chủ xử lý ở đầu ra có thể kết hợp thành một nhóm để cùng nhau đọc dữ liệu. Kafka sẽ tự động chia đều các phân vùng dữ liệu cho từng máy chủ trong nhóm. Nhờ cơ chế này, khi lưu lượng truy cập tăng vọt khiến hệ thống bị quá tải, quản trị viên chỉ cần bổ sung thêm máy chủ mới vào nhóm là khả năng xử lý sẽ lập tức được nhân lên một cách linh hoạt.

2.3 Khung tính toán phân tán Apache Spark

2.3.1 Cơ chế tính toán trực tiếp trên bộ nhớ

Sức mạnh của Apache Spark nằm ở kiến trúc lõi với cấu trúc dữ liệu nền tảng RDD (tập dữ liệu phân tán có khả năng phục hồi). Cấu trúc này cho phép chia nhỏ dữ liệu và phân tán trên bộ nhớ RAM của nhiều máy trạm để tính toán song song. Khác với các công cụ thế hệ cũ thường phải đọc và ghi dữ liệu trung gian liên tục xuống ổ cứng gây chậm trễ, Spark thực hiện phần lớn các tác vụ trực tiếp trên RAM, đem lại tốc độ xử lý vượt trội.

Spark tối ưu hóa lộ trình tính toán bằng cách xây dựng một đồ thị luồng công việc có hướng. Hệ thống sử dụng cơ chế tính toán lười, tức là gom các phép biến đổi lại và chỉ thực thi khi có lệnh xuất kết quả. Kỹ thuật này giúp tiết kiệm tối đa tài nguyên máy tính và loại bỏ các thao tác xử lý dư thừa.

2.3.2 Xử lý luồng có cấu trúc và Khung thời gian

Đồ án áp dụng kỹ thuật xử lý luồng có cấu trúc của Spark. Kỹ thuật này coi luồng dữ liệu đang chảy liên tục như một bảng cơ sở dữ liệu vô hạn, nơi các dòng dữ liệu mới liên

tục được đẩy vào. Hệ thống sẽ áp dụng cơ chế chia dữ liệu thành các lô cực nhỏ để tính toán và cập nhật kết quả liên tục từng giây.

Đặc biệt, Spark cung cấp cơ chế tính toán theo khung thời gian cố định. Thay vì phân tích toàn bộ lịch sử dữ liệu từ trước đến nay, Spark cho phép cắt thời gian thực thành các đoạn ngắn (ví dụ mỗi phút là một khung) để gom nhóm và tổng hợp dữ liệu riêng biệt trong khung đó. Đây là công cụ đắc lực để giám sát sự gia tăng đột biến của các sự kiện an ninh.

2.4 Ứng dụng trong An toàn thông tin

2.4.1 Khái niệm về hệ thống quản lý sự kiện bảo mật

Hệ thống quản lý sự kiện bảo mật (gọi tắt là SIEM) có nhiệm vụ cốt lõi là thu thập nhật ký từ hàng loạt thiết bị phần cứng, máy chủ và ứng dụng trong mạng nội bộ. Sau đó, hệ thống tiến hành đối chiếu các sự kiện rời rạc này lại với nhau để phân tích và phát hiện ra các hành vi thâm nhập bất hợp pháp, cung cấp cái nhìn toàn cảnh về bức tranh an ninh của doanh nghiệp.

2.4.2 Cơ chế phát hiện tấn công dò đoán mật khẩu

Trong an ninh mạng, tấn công dò đoán mật khẩu thường được thực hiện thông qua việc điều khiển hàng ngàn máy tính bị nhiễm mã độc để tự động gửi yêu cầu đăng nhập liên tục vào hệ thống. Dấu hiệu nhận biết rõ nét nhất của hình thức này là tần suất xuất hiện dày đặc của các phản hồi lỗi xác thực (mã lỗi 401) xuất phát từ cùng một địa chỉ IP.

Để tự động hóa việc phát hiện, hệ thống Spark sẽ tiếp nhận luồng dữ liệu định dạng JSON từ máy chủ web, sau đó bóc tách để trích xuất ra hai thông tin then chốt: địa chỉ IP nguồn và mã trạng thái HTTP.

Thuật toán phát hiện được thiết lập dựa trên giới hạn ngưỡng kết hợp với khung thời gian cố định. Cụ thể, hệ thống sẽ cắt thời gian thực thành các khung liên tiếp nhau (ví dụ 1 phút mỗi khung). Bên trong mỗi khung thời gian này, Spark tiến hành gom nhóm dữ liệu theo từng địa chỉ IP và đếm tổng số lần xuất hiện của mã lỗi 401. Nếu số lượt lỗi của một IP cụ thể vượt qua ngưỡng cho trước, hệ thống ngay lập tức kích hoạt cảnh báo và tự động đưa IP đó vào danh sách chặn.

Chương 3

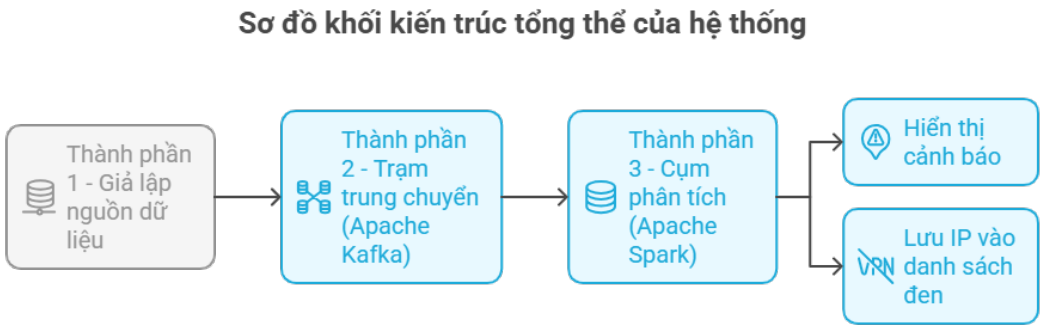
ỨNG DỤNG VÀ TRIỂN KHAI BÀI TOÁN THỰC TẾ

3.1 Phương pháp nghiên cứu và kiến trúc hệ thống

3.1.1 Sơ đồ khối kiến trúc tổng thể

Để hiện thực hóa hệ thống SOC, đồ án thiết kế một chuỗi quy trình xử lý dữ liệu khép kín. Kiến trúc này được chia thành ba thành phần chính, hoạt động nối tiếp nhau theo thời gian thực:

- **Thành phần thứ nhất: Trình tạo dữ liệu giả lập.** Thay vì sử dụng máy chủ thực tế, nghiên cứu phát triển một kịch bản Python đóng vai trò nguồn phát sinh sự kiện. Kịch bản này liên tục tạo ra các bản ghi nhật ký truy cập, trong đó lồng ghép lưu lượng mạng thông thường với các mẫu tấn công dò đoán mật khẩu từ mạng máy tính ma, sau đó truyền tải toàn bộ luồng dữ liệu này đến trạm trung chuyển.
- **Thành phần thứ hai: Trạm trung chuyển Apache Kafka.** Nền tảng này đóng vai trò cửa ngõ tiếp nhận toàn bộ lưu lượng sự kiện từ nguồn phát. Nhờ cơ chế lưu trữ đệm phân tán, Kafka giúp điều hòa tốc độ truyền tải, đảm bảo các máy chủ phân tích phía sau không bị quá tải hoặc ngưng trệ khi lưu lượng tấn công gia tăng đột biến.
- **Thành phần thứ ba: Cụm xử lý phân tích Apache Spark.** Đây là lõi tính toán của toàn bộ kiến trúc, chịu trách nhiệm truy xuất liên tục luồng sự kiện từ Kafka để xử lý trực tiếp trên bộ nhớ. Hệ thống thực hiện các phép toán gom nhóm và thống kê tần suất lỗi xác thực của từng địa chỉ IP theo các khung thời gian thực. Khi phát hiện chỉ số vượt ngưỡng an toàn, Spark sẽ xuất cảnh báo ra màn hình quản trị và tự động cập nhật địa chỉ IP vi phạm vào danh sách chặn.



Hình 3.1: Sơ đồ khối kiến trúc tổng thể của hệ thống

3.1.2 Lựa chọn công nghệ triển khai ảo hóa

Trong môi trường thực tế, việc tích hợp nhiều nền tảng phân tán như Kafka và Spark trên cùng một máy chủ vật lý thường phát sinh xung đột về tài nguyên và thư viện phụ thuộc.

Để giải quyết triệt để vấn đề này, toàn bộ kiến trúc được triển khai dựa trên nền tảng ảo hóa cấp hệ điều hành Docker. Giải pháp này đóng gói từng dịch vụ thành các môi trường thực thi hoàn toàn độc lập, loại bỏ rủi ro can thiệp chéo giữa các tiến trình.

Đồng thời, công cụ quản lý Docker Compose được sử dụng để định nghĩa và liên kết chuỗi dịch vụ. Thông qua một tệp cấu hình tập trung, hệ thống có thể khởi động đồng loạt từ khâu sinh dữ liệu, trung chuyển đến xử lý phân tích. Cách tiếp cận này đảm bảo tính nhất quán cao, cô lập hoàn toàn môi trường thử nghiệm và đáp ứng đúng tiêu chuẩn triển khai vi dịch vụ hiện đại.

3.2 Thu thập và giả lập dữ liệu

3.2.1 Xây dựng kịch bản mạng máy tính ma tấn công

Trong môi trường thử nghiệm, hệ thống không sử dụng máy chủ web thực tế mà vận hành một tập lệnh Python để giả lập quá trình sinh nhật ký truy cập. Dữ liệu đầu ra được định dạng theo chuẩn JSON, bao gồm các trường thông tin trọng yếu để phục vụ công tác phân tích an ninh. Cấu trúc chi tiết của một bản ghi nhật ký được mô tả trong Bảng 3.1.

Trường dữ liệu	**Kiểu dữ liệu**	**Mô tả ý nghĩa**
`ip_address`	String	Địa chỉ IP nguồn của máy khách thực hiện truy cập.
`timestamp`	String	Thời gian ghi nhận sự kiện (định dạng ISO 8601).
`method`	String	Phương thức HTTP được sử dụng (`GET`, `POST`...).
`endpoint`	String	Đường dẫn tài nguyên đích trên máy chủ (ví dụ: `/login`).
`status_code`	Integer	Mã trạng thái phản hồi HTTP (ví dụ: `200`, `401`, `404`).

Bảng 3.1: Cấu trúc các trường thông tin trong bản ghi nhật ký giả lập

Kịch bản sinh dữ liệu được thiết kế bằng cách hòa trộn hai luồng lưu lượng mạng hoạt động song song. Luồng thứ nhất mô phỏng hành vi của người dùng hợp lệ, sinh ra các địa chỉ IP ngẫu nhiên phân bổ rải rác theo thời gian với tỷ lệ trả về mã trạng thái thành công cao.

Luồng thứ hai đóng vai trò cấu thành một cuộc tấn công từ mạng máy tính ma. Tại luồng này, một danh sách cố định các địa chỉ IP độc hại được thiết lập để liên tục gửi yêu cầu truy cập vào đường dẫn đăng nhập (`/login`) của hệ thống. Tần suất của luồng này được đẩy lên mức rất cao, đồng thời chủ đích sinh ra các mã lỗi xác thực (mã lỗi `401`). Sự chênh lệch đột biến về tần suất lỗi từ một nhóm IP cố định chính là đặc điểm nhận dạng cốt lõi để kiểm thử thuật toán phát hiện bất thường.

Dưới đây là ví dụ minh họa trực quan về sự khác biệt giữa hai bản ghi dữ liệu được sinh ra từ hệ thống:

Bản ghi 1: Lưu lượng người dùng hợp lệ (Truy cập thành công)

```
{"ip_address": "192.168.1.45", "timestamp": "2024-05-12T10:05:22Z
   ", "method": "GET", "endpoint": "/home", "status_code": 200}
```

Bản ghi 2: Lưu lượng mạng Botnet (Tấn công dò đoán mật khẩu)

```
{"ip_address": "10.0.0.99", "timestamp": "2024-05-12T10:05:22Z", "
   method": "POST", "endpoint": "/login", "status_code": 401}
```

3.2.2 Cấu hình nhà sản xuất dữ liệu Kafka

Để thiết lập tuyến truyền tải từ tiến trình giả lập đến trạm trung chuyển Kafka, một đối tượng nhà sản xuất dữ liệu (Producer) được khởi tạo thông qua thư viện giao tiếp lập trình. Nhiệm vụ của thành phần này là đảm bảo mọi bản ghi nhật ký đều được đưa vào hệ thống hàng đợi một cách an toàn và đúng trình tự.

Về mặt kỹ thuật, đối tượng này thực hiện thao tác tuần tự hóa (Serialization) toàn bộ các chuỗi JSON thành định dạng luồng byte trước khi truyền qua môi trường mạng. Luồng thông điệp sau đó được đẩy vào một chủ đề chuyên biệt đã được khai báo sẵn trên hệ thống Kafka.

Nhằm mô phỏng chính xác môi trường dữ liệu lớn với áp lực lưu lượng cao, quá trình gửi thông điệp được cấu hình hoạt động theo cơ chế bất đồng bộ. Thay vì tuần tự chờ phản hồi, tiến trình sinh dữ liệu có thể liên tục đẩy hàng ngàn bản ghi mỗi giây mà không bị đình trệ, giúp tối ưu hóa hiệu suất băng thông và ép hệ thống phân tích phía sau phải hoạt động ở trạng thái giới hạn tải.

3.3 Xử lý và phân tích dữ liệu

3.3.1 Truy xuất và tái cấu trúc dữ liệu với Spark (ETL Pipeline)

Sau khi lưu lượng nhật ký được tập kết an toàn tại trạm trung chuyển Kafka, cụm Apache Spark được khởi chạy để đóng vai trò làm cỗ máy xử lý luồng (Stream Processor). Khác với các kiến trúc nguyên khối, hệ thống trong đồ án tách biệt hoàn toàn khâu tiền xử lý dữ liệu và khâu đối sánh tập luật.

Nhiệm vụ đầu tiên của Spark là thiết lập kết nối với Kafka để truy xuất luồng byte thô. Thông qua giao diện lập trình PySpark, một lược đồ dữ liệu (Schema) tĩnh được định nghĩa nghiêm ngặt, bao gồm các trường: địa chỉ IP, đường dẫn, phương thức, mã trạng thái và thời gian.

Khi luồng dữ liệu nhị phân chảy vào, khung tính toán sẽ lập tức chuyển đổi chúng thành chuỗi ký tự và áp dụng lược đồ trên để bóc tách định dạng JSON. Kết quả thu được là một khung dữ liệu phân tán (DataFrame) có cấu trúc chuẩn hóa, sẵn sàng cho các thao tác lưu trữ.

Trích đoạn mã 1: Kết nối Kafka và bóc tách lược đồ JSON

```
spark = SparkSession.builder.appName("SIEM_DataProcessing").
   getOrCreate()
```

```
schema = StructType() \
    .add("ip", StringType()) \
    .add("url", StringType()) \
    .add("method", StringType()) \
    .add("status", IntegerType()) \
    .add("time", TimestampType())

df = spark.readStream.format("kafka").option("kafka.bootstrap.
   servers", "kafka:29092").option("subscribe", "web-logs").load()

parsed_df = df.select(from_json(col("value").cast("string"),
   schema).alias("data")).select("data.*")
```

3.3.2 Cơ chế xử lý lô nhỏ và lưu trữ Hồ dữ liệu (Micro-batching)

Để đảm bảo tốc độ đáp ứng thời gian thực nhưng không làm quá tải thiết bị lưu trữ vật lý, đồ án áp dụng cơ chế chia lô cực nhỏ (Micro-batching) của Spark Structured Streaming.

Thay vì ghi từng dòng nhật ký đơn lẻ xuống ổ cứng, hệ thống sử dụng bộ kích hoạt thời gian (Trigger) để gom toàn bộ dữ liệu nhận được mỗi 3 giây thành một lô. Từng lô dữ liệu này sau đó được đẩy qua hàm xử lý tuần tự để xuất ra các tệp định dạng CSV. Thư mục chứa các tệp này đóng vai trò như một Hồ dữ liệu (Data Lake) cục bộ, liên tục phình to theo thời gian thực để cung cấp nguồn nguyên liệu thô cho phân hệ cảnh báo.

Trích đoạn mã 2: Cơ chế đẩy dữ liệu lô nhỏ xuống Hồ dữ liệu

```
def process_batch(batch_df, batch_id):
    batch_df.write.format("csv").mode("append").option("header", "
       true").save("/app/ket_qua_output")

query = parsed_df.writeStream.outputMode("append").foreachBatch(
   process_batch).trigger(processingTime='3␣seconds').start()
query.awaitTermination()
```

3.3.3 Thuật toán phân tích và phát hiện bất thường

Dựa trên nền tảng dữ liệu liên tục được cập nhật từ Hồ dữ liệu, một phân hệ phân tích độc lập được xây dựng nhằm quét và phát hiện các luồng tấn công. Việc tách rời cụm tính toán Spark và phân hệ phân tích giúp hệ thống đạt độ khả dụng cao: ngay cả khi giao diện giám sát bị gián đoạn, luồng dữ liệu vẫn không bị thất thoát.

Thuật toán nhận diện sự cố được lập trình dựa trên tập luật đối sánh ngưỡng (Threshold-based Rule Matching), phân loại các mối đe dọa thành ba nhóm kịch bản chính:

1. **Tấn công dò đoán mật khẩu:** Hệ thống lọc các bản ghi trả về mã lỗi 401 (Lỗi xác thực), sau đó tiến hành gom nhóm theo địa chỉ IP nguồn. Nếu một địa chỉ IP phát sinh lớn hơn 10 lỗi, hệ thống lập tức gắn cờ cảnh báo mạng máy tính ma.

2. **Dò quét lỗ hổng hệ thống:** Thuật toán lọc các mã lỗi 404 (Không tìm thấy tài nguyên). Việc một IP liên tục truy cập vào các đường dẫn ẩn và nhận về hơn 10 mã lỗi 404 được hệ thống xác định là hành vi rà quét thư mục quản trị trái phép.

3. **Thu thập dữ liệu tự động:** Đối với các truy cập trả về mã 200 (Thành công), hệ thống vẫn tiến hành thống kê. Nếu tần suất tải trang của một IP vượt quá ngưỡng 30 lần trong một khung thời gian ngắn, IP đó sẽ bị đưa vào danh sách theo dõi vì có dấu hiệu sử dụng công cụ tự động để cào dữ liệu nghiệp vụ.

Thông qua việc kết hợp các phép toán xử lý luồng tại Spark và đối sánh tập luật linh hoạt, hệ thống duy trì được mức độ chính xác cao trong việc nhận diện đa dạng các mẫu hình tấn công, đồng thời loại bỏ được các cảnh báo sai gây ra bởi sự nhầm lẫn của người dùng thông thường.

Chương 4

KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Kết quả chạy thực nghiệm

4.1.1 Giao diện sinh dữ liệu log

(Nội dung...)

4.1.2 Bảng cảnh báo thời gian thực và File kết quả

(Nội dung...)

4.2 Đưa ra nhận xét và đánh giá

4.2.1 Ưu điểm của hệ thống

(Nội dung...)

4.2.2 Hạn chế và hướng phát triển

(Nội dung...)

KẾT LUẬN

(Nội dung kết luận...)

LỜI CẢM ƠN

(Nội dung lời cảm ơn...)

TÀI LIỆU THAM KHẢO

(Liệt kê tài liệu...)